Amazon RDS PostgreSQL Production Setup – Multi-AZ Cluster with Writer and Reader Endpoints

To Begin with the Lab

Summary of the Lab

In this lab, a production-style **Amazon RDS** PostgreSQL cluster was created using the **Production** template instead of the earlier free-tier setup. The cluster deployed one writer instance and two reader instances across different Availability Zones so the database could support both high availability and read scaling. A jump server was used to connect to the database with the PostgreSQL client, and sample database objects were created on the writer endpoint. The reader endpoint was then tested to confirm that it allowed read operations but rejected write attempts. Finally, the lab verified that the cluster exposed separate writer and reader endpoints and could fail over automatically if the primary zone became unavailable.

Understanding Amazon RDS Multi-AZ Cluster with Writer and Reader Endpoints

A production **Amazon RDS** Multi-AZ cluster exists so a database can stay available even when one Availability Zone has a problem. The writer endpoint sends all insert, update, and delete activity to the primary writer instance, while the reader endpoint distributes read queries across the reader nodes. This setup improves performance because reads and writes are separated instead of all traffic hitting one instance. It also improves availability because AWS can promote a reader node if the writer-side Availability Zone fails.

Example:

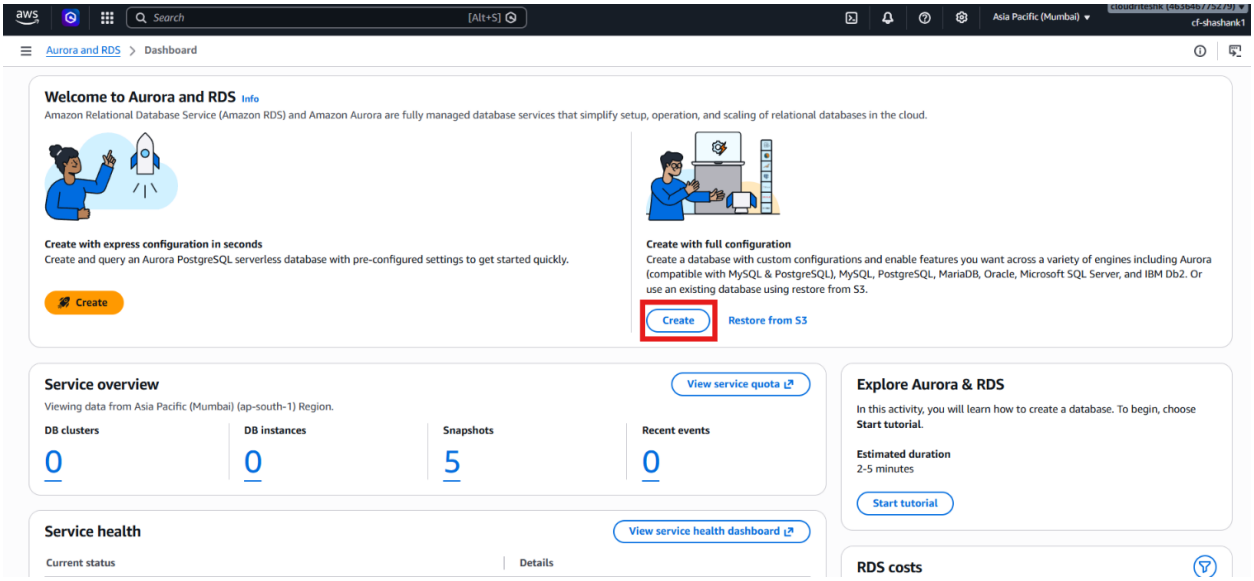
An application writes new orders through the writer endpoint and runs reporting queries through the reader endpoint. If the writer's Availability Zone fails, AWS automatically promotes a reader node and keeps the application online. That means the app continues working without manual intervention or downtime.

Lab Steps

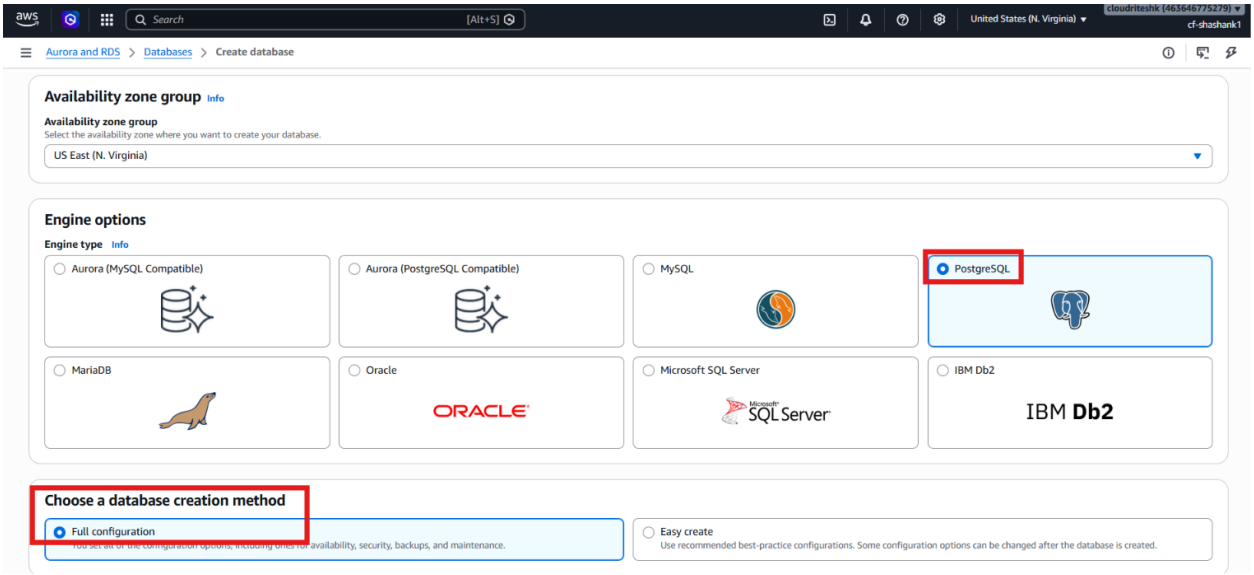
Step 1 – Create the Production Cluster

Create a PostgreSQL RDS cluster using the **Production** template and the Multi-AZ cluster option.

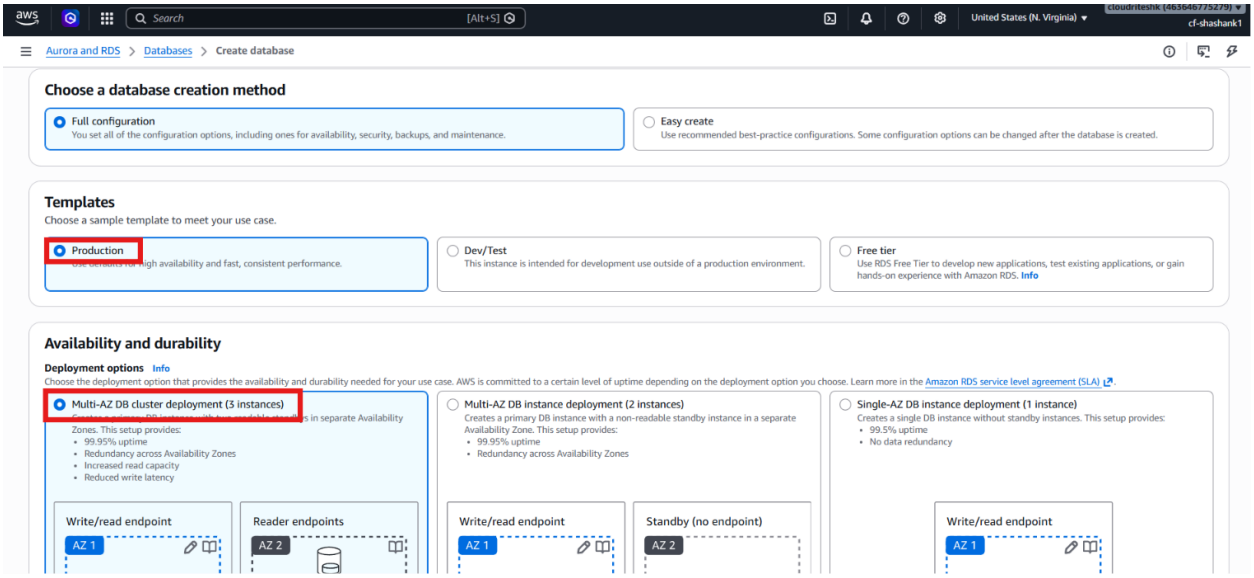
- Sign in to the AWS Management Console.
- Open **Amazon RDS**.
- Below Create with full configuration and Click **Create.**



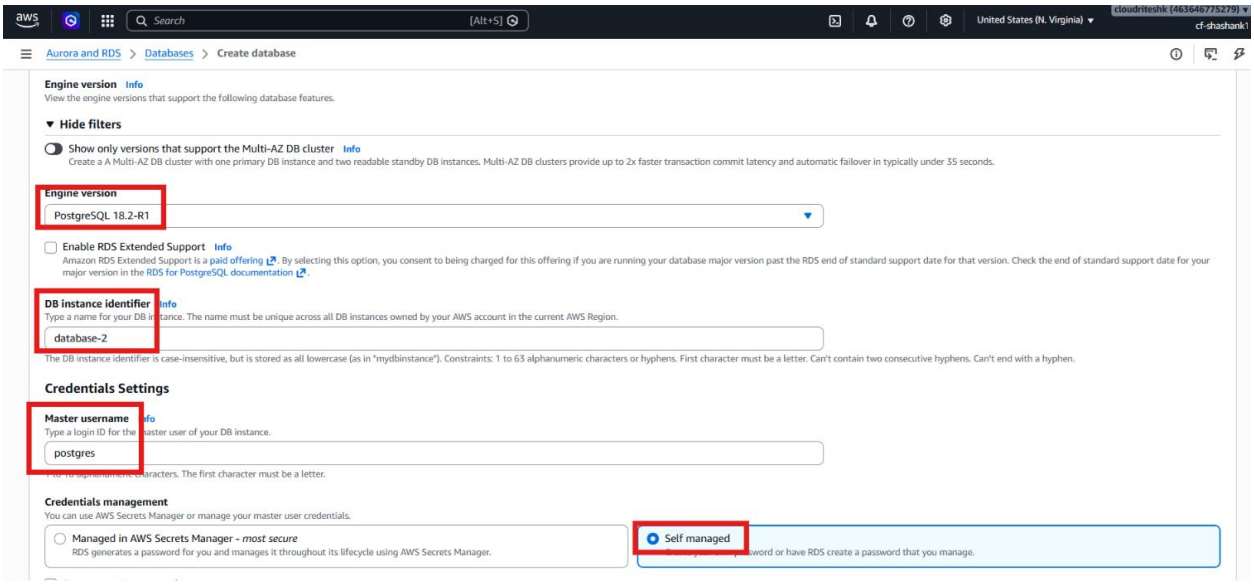
- Select any engine but for this lab I have chosen **PostgreSQL**.



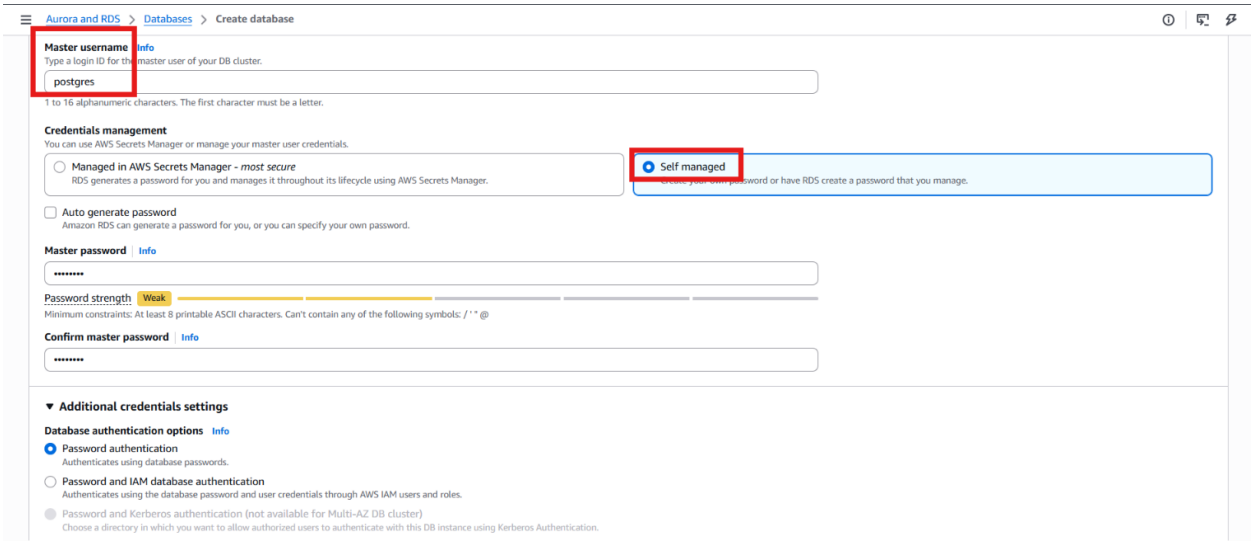
- Choose **Full Configuration**.
- Select the **Production** template.
- Choose the Multi-AZ cluster deployment option with one writer and two reader instances (3 instances).



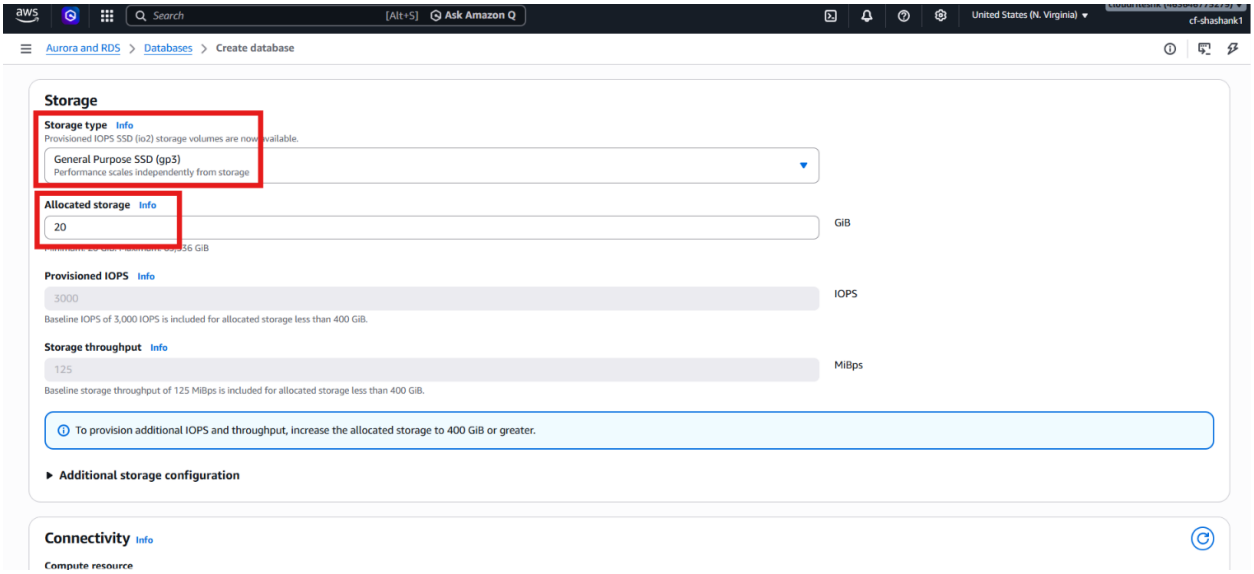
- Choose correct PostgreSQL Version.



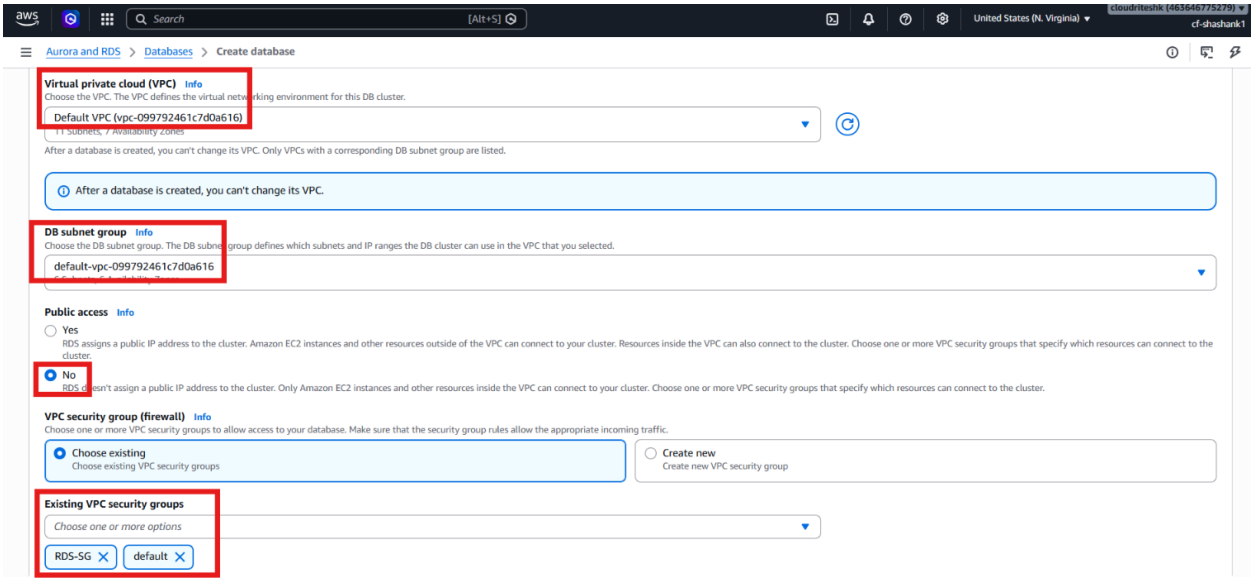
- Enter the database identifier and credentials.



- Select a suitable instance class and storage size but for lab purpose we are selecting lower ones as shown.



- Let AWS auto-generate the password or set one manually.
- Choose the appropriate VPC and database subnet group.
- Keep **Public access** set to **No**.
- Attach the RDS security group created for the lab.

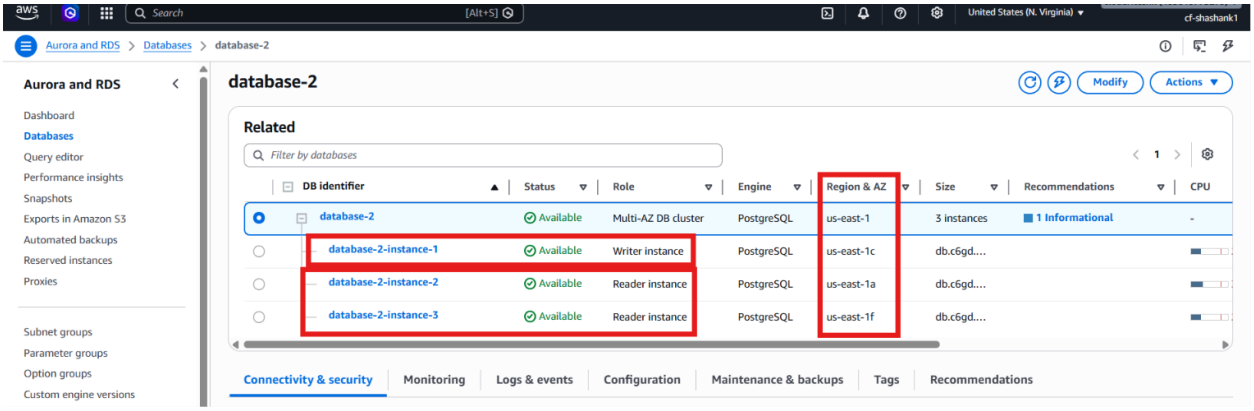


- Review the estimated monthly cost.
- Click **Create database**.

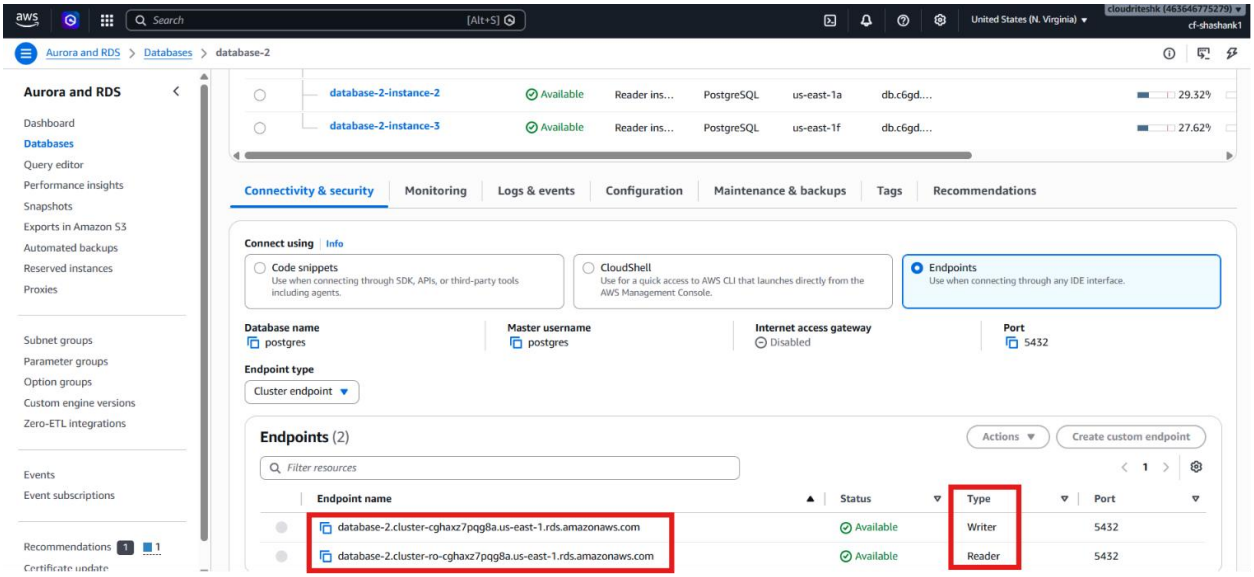
Step 2 – Wait for the Cluster and Verify the Topology

Wait until the cluster becomes available and confirm that the writer and reader nodes are placed in different Availability Zones.

- Wait for the cluster status to change from **Creating** or **Modifying** to **Available**.
- Open the cluster details page.
- Verify that the cluster contains one writer instance and two reader instances.
- Confirm that the configuration shows **Multi-AZ = Yes**.
- Check that the writer and reader nodes are spread across different Availability Zones.

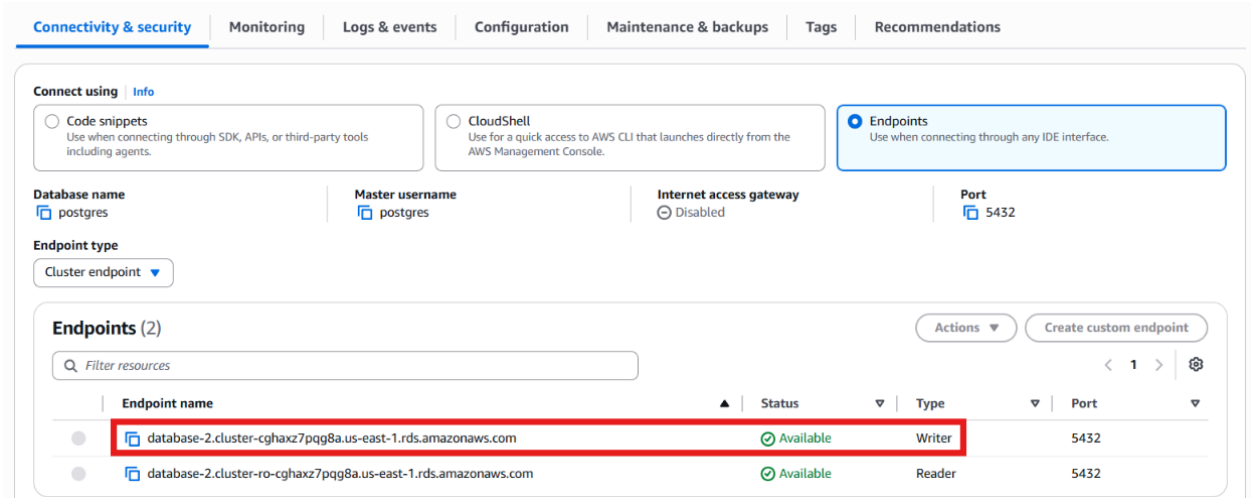


- Note the writer endpoint and reader endpoint shown in the console.

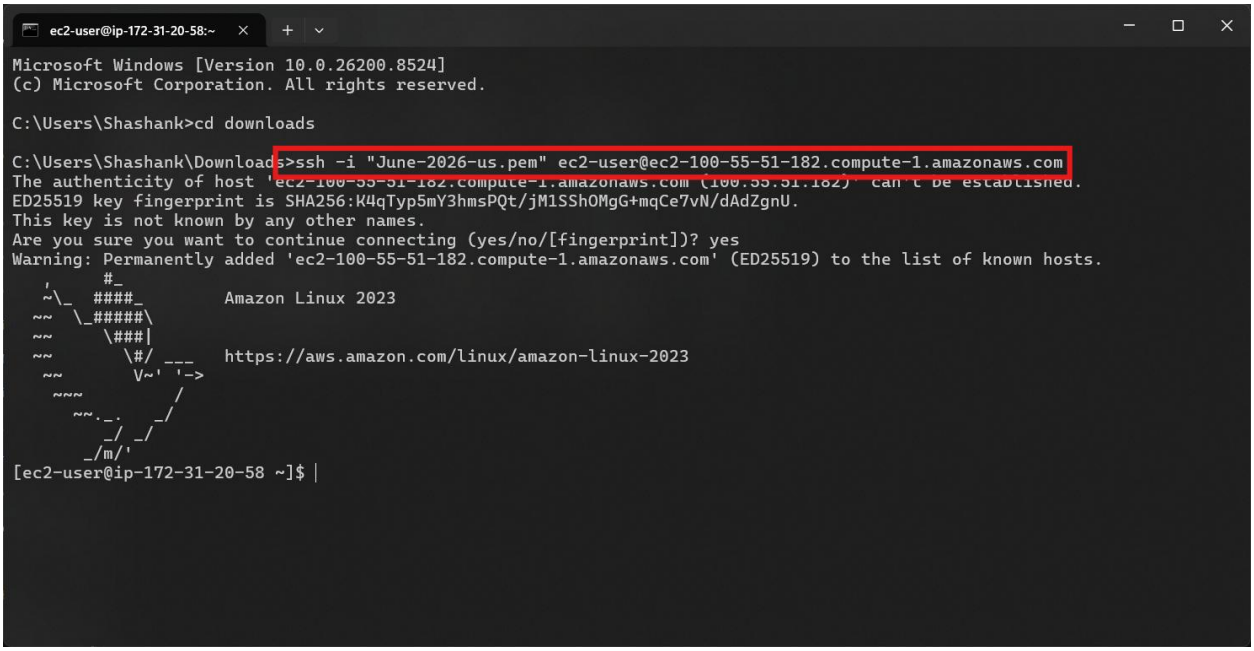


Step 3 – Connect to the Writer Endpoint from the Jump Server

Use the jump server and PostgreSQL client to connect to the writer endpoint, then create a sample database and table.



- Open the jump server.
- Connect to the jump server using SSH.



- Use the PostgreSQL client installed on the server.
- If not, then install the PostgreSQL server via the following commands.

```
sudo -i
sudo dnf update -y
sudo dnf install -y postgresql18
```

```
root@ip-172-31-20-58:~
[root@ip-172-31-20-58 ~]# sudo dnf update
Last metadata expiration check: 0:01:56 ago on Wed Jun 17 09:41:59 2026.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-20-58 ~]# sudo dnf install -y postgresql18
Last metadata expiration check: 0:02:10 ago on Wed Jun 17 09:41:59 2026.
Dependencies resolved.
================================================================================
 Package                    Architecture   Version                  Repository        Size
================================================================================
Installing:
 postgresql18               x86_64         18.4-1.amzn2023.0.1      amazonlinux      2.0 M
Installing dependencies:
 postgresql18-private-libs  x86_64         18.4-1.amzn2023.0.1      amazonlinux      160 k

Transaction Summary
================================================================================
Install  2 Packages

Total size: 2.1 M
Installed size: 8.9 M
Downloading Packages:
[SKIPPED] postgresql18-18.4-1.amzn2023.0.1.x86_64.rpm: Already downloaded
[SKIPPED] postgresql18-private-libs-18.4-1.amzn2023.0.1.x86_64.rpm: Already downloaded
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Installing       : postgresql18-private-libs-18.4-1.amzn2023.0.1.x86_64   1/2
  Installing       : postgresql18-18.4-1.amzn2023.0.1.x86_64                2/2
  Running scriptlet: postgresql18-18.4-1.amzn2023.0.1.x86_64                2/2
  Verifying        : postgresql18-18.4-1.amzn2023.0.1.x86_64                1/2
  Verifying        : postgresql18-private-libs-18.4-1.amzn2023.0.1.x86_64   2/2

Installed:
  postgresql18-18.4-1.amzn2023.0.1.x86_64
  postgresql18-private-libs-18.4-1.amzn2023.0.1.x86_64

Complete!
```

- Connect to the writer endpoint using the master credentials.

- psql -h {endpoint} -U {username}

```
root@ip-172-31-20-58:~
[root@ip-172-31-20-58 ~]# psql -h database-2.cluster-cghaxz7pqg8a.us-east-1.rds.amazonaws.com -U postgres
Password for user postgres:
psql (18.4, server 18.2)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

postgres=>
```

- Create a sample database.
- Switch to the new database.
- Create a sample table for testing.

```
CREATE DATABASE demo;

\c demo

CREATE TABLE employees (
   id SERIAL PRIMARY KEY,
```

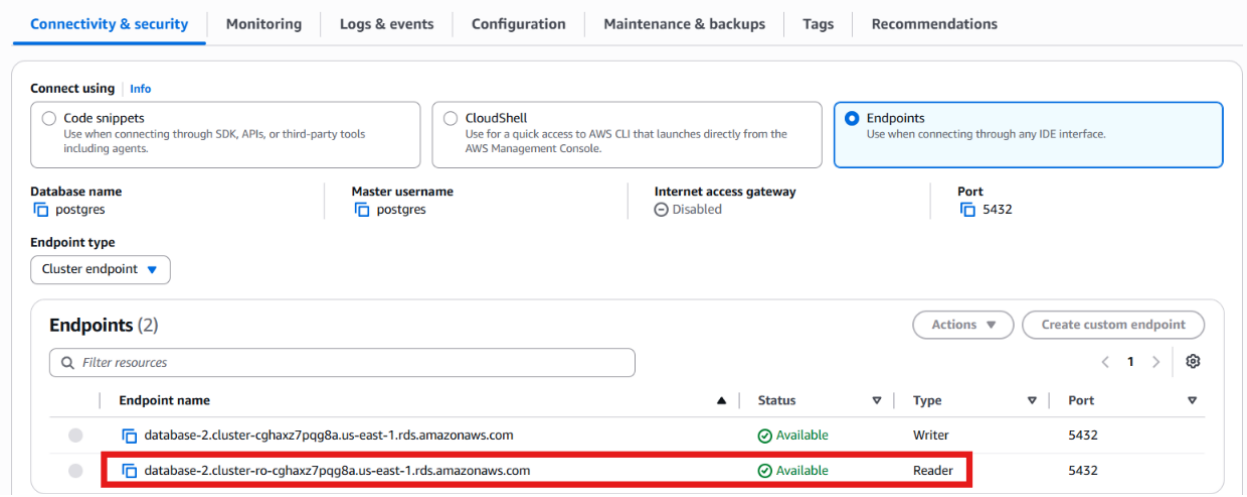
```
    name VARCHAR(100) NOT NULL,
    position VARCHAR(100)
);

INSERT INTO employees (name, position)
VALUES ('John Doe', 'Software Engineer');

INSERT INTO employees (name, position)
VALUES ('Jane Smith', 'Project Manager');

SELECT * FROM employees;
```

```
postgres=> CREATE DATABASE demo;
CREATE DATABASE
postgres=> \c demo
psql (18.4, server 18.2)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
You are now connected to database "demo" as user "postgres".
demo=> CREATE TABLE employees (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    position VARCHAR(100)
);
CREATE TABLE
demo=> INSERT INTO employees (name, position)
VALUES ('John Doe', 'Software Engineer');
INSERT 0 1
demo=> INSERT INTO employees (name, position)
VALUES ('Jane Smith', 'Project Manager');
INSERT 0 1
demo=> SELECT * FROM employees;
 id |    name    |     position
----+------------+------------------
  1 | John Doe   | Software Engineer
  2 | Jane Smith | Project Manager
(2 rows)

demo=>
```

- Verify that the write operations succeed on the writer endpoint.

Step 4 – Test the Reader Endpoint

Connect to the reader endpoint and confirm that it is read-only.



- Open a new PostgreSQL client session.
- Connect to the reader endpoint instead of the writer endpoint.

```
[root@ip-172-31-20-58 ~]# psql -h database-2.cluster-ro-cghaxz7pqg8a.us-east-1.rds.amazonaws.com -U postgres
Password for user postgres:
psql (18.4, server 18.2)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

postgres=>
```

- Change database to demo.
- Run a SELECT query on the sample table.

```
SELECT * FROM employees;
```

- Verify that the data is returned successfully.
- Attempt an INSERT statement on the reader endpoint.

```
INSERT INTO employees (name, position)
VALUES ('Test User', 'Analyst');
```

- Confirm that the write attempt fails with a read-only transaction error.

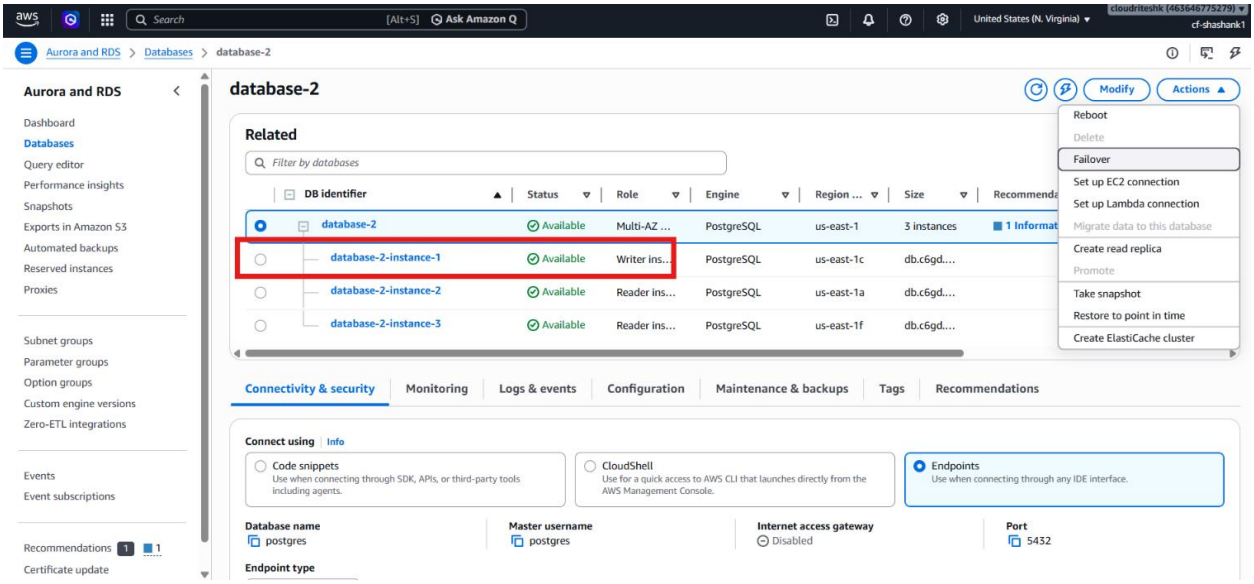
```
[root@ip-172-31-20-58 ~]# psql -h database-2.cluster-ro-cghaxz7pqg8a.us-east-1.rds.amazonaws.com -U postgres
Password for user postgres:
psql (18.4, server 18.2)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

postgres=> \c demo
psql (18.4, server 18.2)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
You are now connected to database "demo" as user "postgres".
demo=> SELECT * FROM employees;
 id |    name    |     position
----+------------+------------------
  1 | John Doe   | Software Engineer
  2 | Jane Smith | Project Manager
(2 rows)

demo=> INSERT INTO employees (name, position)
VALUES ('Test User', 'Analyst');
ERROR:  cannot execute INSERT in a read-only transaction
demo=>
```

- Verify that the reader endpoint is intended only for read traffic.

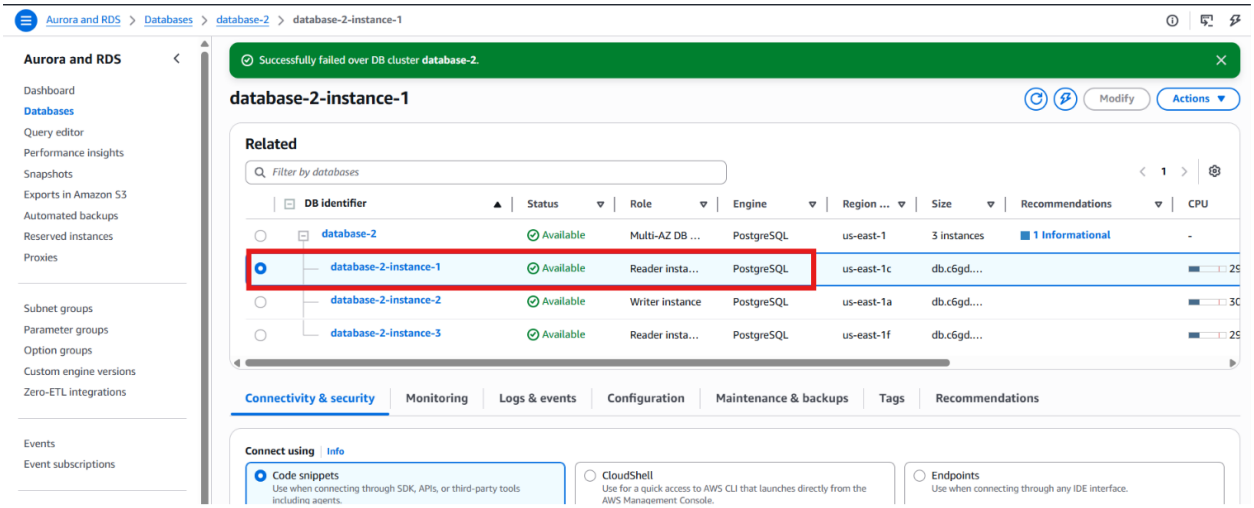
Step 5 – Review Failover and Clean Up

Review how AWS handles failover automatically and then remove the cluster after the demo.

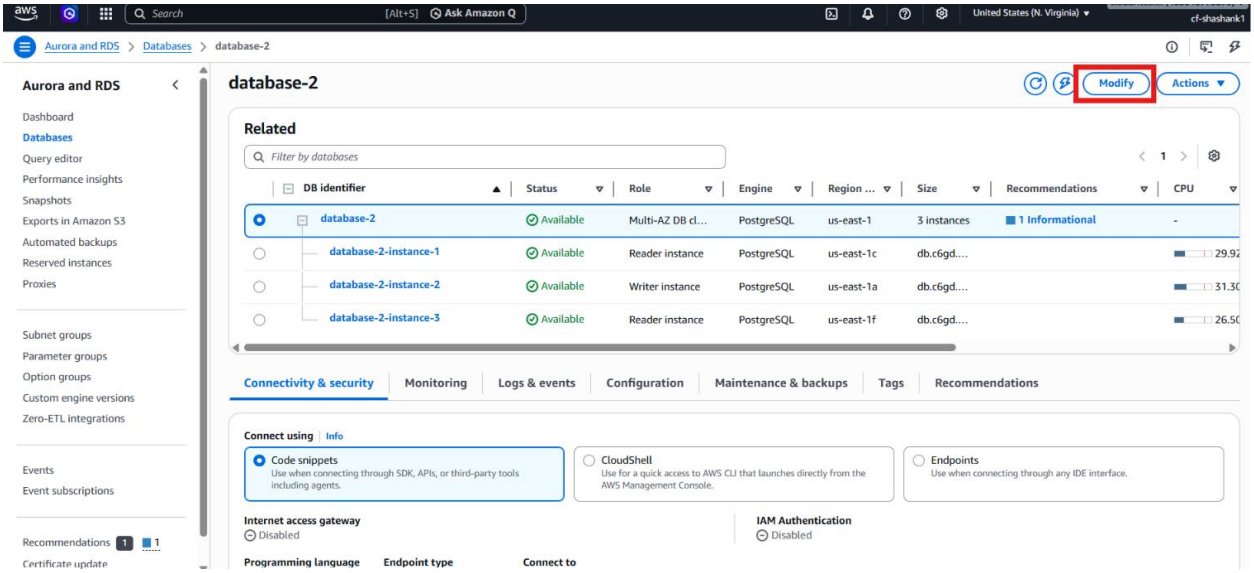
- Before failover database-2-instance-1 was writer instance.



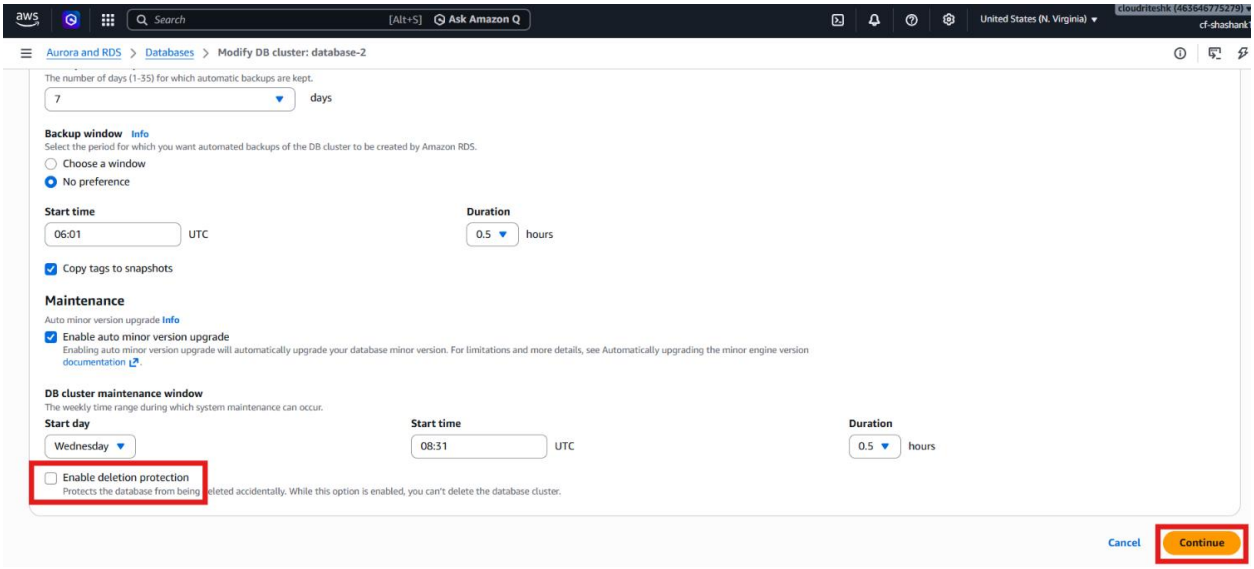
- After failover AWS picked instance-2 to become the new writer automatically (promoted) and the instance-1 demoted to reader.



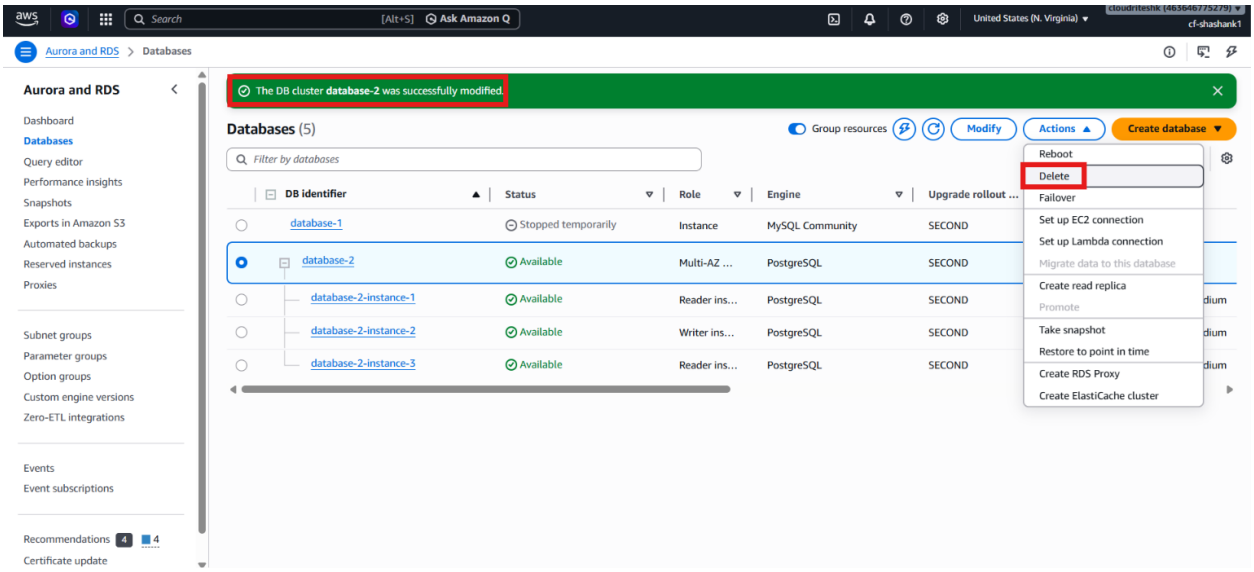
- Select database and click on **Modify**.



- Scroll down and disable deletion protection if it is enabled.



- Wait for the change to apply.
- Delete the cluster after the lab is complete.



- Confirm that the cluster is removed successfully.

Verification

- The PostgreSQL RDS cluster was created successfully.
- The cluster used the **Production** template.
- The cluster included one writer node and two reader nodes.
- The writer endpoint accepted write operations.
- The reader endpoint allowed reads and rejected writes.
- The cluster was configured for Multi-AZ high availability.